

Pandav Backend V2 - System Design and Social Network Plan

This document defines a scalable and secure backend v2 design for Pandav, with role-based workflow, social feed, professional networking, personal productivity, and chat.

1. Vision

Pandav v2 keeps all v1 core features (auth, teams, tasks, attendance, admin controls, chat) and adds a social collaboration layer where users can:

- Publish work updates as posts.
- Build internal professional connections.
- Chat with approved connections.
- Manage personal todo tasks and publish task progress.
- Request approval for external publishing (LinkedIn and GitHub style workflow).

2. Product Goals

- Keep v1 stable while v2 evolves safely.
- Enforce strict role and team boundaries.
- Add social feed and networking without weakening security.
- Support regular users (non-leader) with personal task and post features.
- Support future scaling with background jobs and modular services.

3. Active API Versioning

- v1 routes: `/api/v1/*`
- v2 routes: `/api/v2/*`
- Legacy alias still available temporarily: `/api/*` for v1 compatibility

4. Role Responsibilities

4.1 System Admin

- Full system control.
- Manage users, teams, moderation, and policy.
- Approve or reject pending external post requests.
- Review audit logs and security events.

4.2 Team Leader

- Create and manage team operations.
- Assign team tasks and review completion.
- Provide feedback on work.
- Approve team member and user external publishing requests (policy scope).
- Participate in direct network chat and team discussions.

4.3 Team Member

- Complete assigned tasks.
- Create personal tasks.
- Publish internal updates.
- Submit LinkedIn/GitHub style publish requests when required.
- Build connections and chat with accepted connections.

4.4 Regular User (Non Leader)

- Not a team leader.
- Can create personal todo tasks and complete them.
- Can create posts and publish to Pandav feed.
- Can generate post content from completed personal tasks.
- Can target pandav , linkedin , github , or any combination.
- Can message another user only with permission (accepted connection), unless admin/leader policy bypass applies.

5. Current V2 Social Module (Implemented)

Mounted at: `/api/v2/social`

5.1 Feed and Posts

- GET /feed - Returns visible feed posts for the current user.
- GET /posts/me - Returns current user posts.
- POST /posts - Create post with targets.
- POST /posts/from-task/:taskId - Generate post from completed personal task.
- POST /posts/:postId/like - Toggle like.
- POST /posts/:postId/comments - Add comment.
- PATCH /posts/:postId/approve - Admin/leader approval endpoint.

Post behavior:

- Visibility options: public , team , connections .
- Posting targets: pandav , linkedin , github .
- Team members and regular users require approval for external targets (linkedin/github).
- Leaders/admin can approve pending posts.

5.2 Personal Tasks (Todo)

- POST /personal-tasks - Create personal task.
- GET /personal-tasks - List your tasks.
- PATCH /personal-tasks/:taskId - Update title/description/status/priority/dueDate.
- PATCH /personal-tasks/:taskId/toggle-complete - Quick complete/uncomplete.
- DELETE /personal-tasks/:taskId - Delete personal task.

5.3 Networking

- GET /users/discover?q= - Find active users by name/email.
- POST /connections/request - Send connection request.
- PATCH /connections/:connectionId/respond - Accept or reject request.
- GET /connections - Get sent, received, and accepted connections.

5.4 Network Chat and Team Group Chat

- POST /messages - Send direct message.
- GET /messages/:userId - Read direct message thread.

Chat behavior:

- Messaging is allowed for accepted connections.
- Admin and team leader can bypass connection gate for operational reasons.

- Real-time event is emitted over [Socket.IO](#) to the receiver room.

5.5 Social Integrations

- POST /integrations/:provider/link - Connect LinkedIn/GitHub account with token verification.
- GET /integrations - List connected providers for current user.

Note:

- Current integration endpoints store connection metadata and are ready for OAuth worker integration.
- External post publishing queue worker is the next step.

6. Data Models Used by V2 Social Module

6.1 SocialPostV2

- author
- sourceTask (optional)
- title
- content
- mediaUrls[]
- tags[]
- visibility
- targets.pandav
- targets.linkedin
- targets.github
- approvalStatus
- approvedBy
- approvedAt
- likes[]
- comments[]
- timestamps

6.2 NetworkConnectionV2

- requester
- recipient
- status (pending , accepted , rejected)

- `respondedAt`
- `timestamps`

6.3 PersonalTaskV2

- `user`
- `title`
- `description`
- `status` (`todo` , `in_progress` , `completed`)
- `priority` (`low` , `medium` , `high`)
- `dueDate`
- `completedAt`
- `timestamps`

6.4 SocialIntegrationV2

- `user`
- `provider` (`linkedin` , `github`)
- `accountHandle`
- `isConnected`
- `scopes[]`
- `accessTokenHint`
- `connectedAt`
- `timestamps`

7. Security Principles for V2

- Authentication required for all social routes (`protect` middleware).
- Role checks for approval operations.
- Visibility checks before reading/interacting with posts.
- Connection checks before user-to-user messaging.
- Schema-level validation (`maxLength` , `required` fields, `enums`).

Recommended hardening next:

- Add request validation middleware (Joi/Zod) for all payloads.
- Add rate limits for auth, messaging, and post creation.
- Add encrypted token vault for OAuth provider tokens.
- Add security event audit trail for approvals and moderation.

8. Scalability Plan

Current design supports iterative growth. For production-scale rollout, add:

- Queue workers for external publishing and notifications.
- Redis adapter for [Socket.IO](#) horizontal scaling.
- Read-optimized feed strategies.
- Caching for discover/profile lookups.
- Full-text index strategy for post search.

9. Suggested Folder Structure (Target)

`src/routes/*` for route namespaces (v2 files use `*v2*.js` naming).

`src/controllers/*` for v2 business logic orchestration (v2 files use `*v2*.js` naming).

`src/models/*` for new domain models (v2 files use `*v2*.js` naming).

As modules expand, introduce:

- `src/services/v2/*`
- `src/repositories/v2/*`
- `src/policies/v2/*`

10. Next Implementation Milestones

1. Add strict request validation middleware on all v2 social endpoints. (Implemented)
2. Add audit logs for post approvals, task changes, and connection actions. (Implemented)
3. Add team group chat model and routes. (Implemented)
4. Add OAuth connect flow and external publisher worker for LinkedIn/GitHub. (Implemented)
5. Add moderation endpoints for admin content review. (Implemented)
6. Add integration tests for feed visibility and connection-gated messaging. (Implemented)

11. Notes for Frontend Team

Primary new base for social features:

- `/api/v2/social/feed`
- `/api/v2/social/posts`
- `/api/v2/social/personal-tasks`

- `/api/v2/social/connections`
- `/api/v2/social/messages`
- `/api/v2/social/integrations`

UI can now build:

- Home feed (post cards + like/comment).
- Discover people.
- Connection requests inbox.
- 1:1 network chat.
- Personal todo list.
- Generate post from completed task.
- Pending approval badge for external publish requests.

12. Backward Compatibility

- v1 remains fully operational on `/api/v1/*`.
- Existing clients can continue v1 usage while v2 social features are incrementally adopted.

13. LinkedIn and GitHub Linking + Fetch + Publish (Implemented)

Link account

- `POST /api/v2/social/integrations/linkedin/link`
- `POST /api/v2/social/integrations/github/link`

Request body example:

```
{
  "accessToken": "provider_access_token",
  "refreshToken": "optional_refresh_token",
  "tokenExpiresAt": "2026-12-31T00:00:00.000Z",
  "scopes": ["read", "write"]
}
```

Fetch linked profile

- GET /api/v2/social/integrations/linkedin/profile
- GET /api/v2/social/integrations/github/profile

Fetch provider posts/activity

- GET /api/v2/social/integrations/linkedin/posts
- GET /api/v2/social/integrations/github/posts

Publish Pandav post to providers

- POST /api/v2/social/posts/:postId/publish

Request body example:

```
{
  "targets": {
    "pandav": true,
    "linkedin": true,
    "github": false
  },
  "githubRepo": "owner/repo"
}
```

Notes:

- If approvalStatus is pending or rejected, external publish is blocked.
- GitHub publish uses Issue creation in target repo.
- LinkedIn publish uses UGC post API.
- Keep provider tokens secure; migrate to encrypted vault before production.

14. Automatic LinkedIn Post From Completed Tasks (Implemented)

When a personal task moves to `completed`, the backend now auto-generates a social post from the task.

Behavior:

- Creates a post with targets: `pandav=true` , `linkedin=true` , `github=false` .
- If user role requires approval for external posting (`user` , `team_member`), post is created as `pending` .
- If approval is not required and LinkedIn is linked, backend attempts immediate publish to LinkedIn.
- Duplicate generation is prevented for the same task.

Task fields added:

- `autoGenerateLinkedInPost` (default `true`)
- `lastAutoGeneratedPost`
- `autoPostStatus`
- `autoPostMessage`

How to control:

- On task creation (`POST /api/v2/social/personal-tasks`), pass `autoGenerateLinkedInPost: false` to disable per task.
- Completing via `PATCH /api/v2/social/personal-tasks/:taskId` with `status=completed` triggers auto flow.
- Completing via `PATCH /api/v2/social/personal-tasks/:taskId/toggle-complete` triggers auto flow.

15. OAuth Flow and Async Publish Jobs (Implemented)

OAuth (LinkedIn/GitHub)

- `GET /api/v2/social/integrations/:provider/oauth/start`
 - Returns provider authorization URL and state token.
- `POST /api/v2/social/integrations/:provider/oauth/callback`
 - Body: `{ "code": "...", "state": "..." }`
 - Exchanges code for access token, fetches provider profile, stores integration.

Async Publish Jobs

- `POST /api/v2/social/posts/:postId/publish/jobs`
 - Queue provider publish jobs (`linkedin` , `github`) for a post.
- `GET /api/v2/social/publish-jobs`
 - List publish jobs (owner scope, admin can view all).

- POST /api/v2/social/publish-jobs/:jobId/process
 - Process one queued/failed job immediately.
- POST /api/v2/social/publish-jobs/:jobId/retry
 - Requeue a failed job.
- POST /api/v2/social/publish-jobs/process-due
 - Process queued jobs whose retry time is due (admin).

Behavior:

- Jobs track queued -> processing -> completed/failed with retry backoff.
- External publish failures are captured in job error and post publish results.
- Audit events are logged for job creation/processing/retry and OAuth completion.

16. Moderation and Integration Test Coverage (Implemented)

Admin Moderation Endpoints

- GET /api/v2/social/moderation/posts
 - Query: status=all|clean|flagged|hidden|removed , optional owner=<userId>
- PATCH /api/v2/social/moderation/posts/:postId
 - Body: { "action": "flag|unflag|hide|unhide|remove|restore", "reason": "optional" }

Moderation behavior:

- Feed excludes removed posts for normal users.
- hidden posts are visible only to owner and admin.
- Moderation changes are audited.

Integration Tests

- File: test/integration/socialV2.integration.test.mjs
- Run command: npm run test:integration
- Covered flows:
 - Feed visibility for connections-only posts
 - Connection-gated direct messaging
 - External publish block while approval is pending

Test prerequisites:

- Backend server running and reachable.
- MongoDB configured.
- Optional env override: `INTEGRATION_BASE_URL` (default: `http://localhost:5000/api/v2`).